# Unidad 3: Gestión integrada de proyectos

Materia: Gestión de configuración del software

Ing. Francisco Álvarez Solís, MSc.

## Objetivo general de la unidad (en simple)

Aprender a **manejar dependencias** (librerías que tu proyecto necesita) y a usar herramientas que ayuden a organizar un proyecto y comunicar avances (incluyendo **Maven** en Java), sin enredarnos con tecnicismos.

# ¿Qué deberías poder hacer al final?

- Explicar con tus palabras qué es una **dependencia** y por qué se debe gestionar.
- Identificar problemas típicos: **"en mi PC funciona"**, conflictos de versiones y dependencias ocultas.
- Usar una herramienta de gestión de proyecto para mostrar avances a un cliente (con evidencias simples).
- Crear y construir un proyecto Java con **Maven** usando comandos básicos.

## Regla de oro de esta unidad

Si tu proyecto no puede construirse en otra máquina siguiendo pasos claros, **no está bien gestionado**.

# Diccionario rápido 1 (para no perderse)

| Término | Significa (en simple) |
|---|---|
| IDE | Editor inteligente para programar (ej. VS Code, IntelliJ). |
| CLI/Terminal | Ventana para escribir comandos (PowerShell, Git Bash, Terminal Linux). |
| Repositorio (repo) | Lugar donde se guardan versiones del código (ej. GitHub). |
| Commit | "Guardar un cambio" con un mensaje en Git. |
| PR (Pull Request) | Solicitud para unir cambios (revisión antes de mezclar). |
| Build | Proceso para construir el software (compilar, probar, empaquetar). |

*No memorices: vuelve a estas láminas cuando veas una palabra rara.*

# Diccionario rápido 2 (muy usado en esta unidad)

| Término | Significa (en simple) |
|---|---|
| JDK | Kit para programar en Java (incluye compilador). |
| CI/CD | Automatización: construir y probar cada vez que se sube código (a veces también despliega). |
| Dependencia | Librería/paquete que tu proyecto necesita (como "ingredientes"). |
| SemVer | Forma común de versionar: MAJOR.MINOR.PATCH (ej. 2.3.1). |
| Artefacto | Archivo final generado (ej. .jar en Java). |
| RBAC | Roles y permisos (quién puede ver/editar; ej. cliente solo comenta). |

# Ruta de aprendizaje (con tareas y foros)

1. **3.1** Gestión de dependencias (qué es, cómo funciona) + **tarea** + **foro**
2. **3.2** Importancia (calidad, seguridad, equipo) + **tarea** + **foro**
3. **3.3** Gestión integrada de proyectos y cliente + **tarea** + **foro**
4. **Maven** (Java) paso a paso: desde crear proyecto hasta empaquetar + **tarea** + **foro**

# 3.1 ¿Qué es una dependencia? (analogía fácil)

- Imagina que tu proyecto es una **receta de cocina**.
- Las **dependencias** son los **ingredientes** (harina, huevos, azúcar).
- Si un ingrediente falta, la receta falla.
- Si cambias un ingrediente por otro (otra marca/versión), el sabor puede cambiar o dañarse.

**Ejemplo real**

Si tu programa necesita leer JSON, probablemente usa una librería (dependencia) para hacerlo.

# 3.1 ¿Por qué “en mi PC funciona”? (caso típico)

- Estudiante A: instaló librerías “a mano” o su PC ya las tenía.
- Estudiante B: abre el proyecto, pero esas librerías no están.
- Resultado:
  - el proyecto no compila,
  - o compila pero falla al ejecutar.

## Solución simple

Que el proyecto tenga una **lista oficial** de dependencias, y que una herramienta las descargue automáticamente.

# 3.1 ¿Qué hace un gestor de dependencias? (paso a paso)

1. Tú escribes en un archivo: "necesito la librería X versión Y".
2. La herramienta busca esa librería en un **repositorio** (catálogo en internet o interno).
3. La descarga y la guarda en tu PC (cache local).
4. Construye el proyecto usando esas librerías.
5. Si otro compañero hace lo mismo, obtiene **lo mismo**.

## Ejemplos de herramientas

Java: Maven/Gradle | JavaScript: npm | Python: pip

# 3.1 Repositorio (repo) de dependencias: ¿qué es?

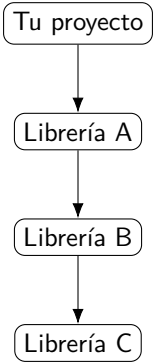
- Es como una **biblioteca**:
  - guardan miles de librerías (paquetes) ya listos,
  - cada librería tiene versiones (ediciones).
- Hay repositorios públicos (internet) y privados (de una empresa/universidad).

**Ejemplo visual**

Tú pides: "Libro X, edición 2.1.0" → el repo te lo entrega.

# 3.1 Dependencias transitivas (las "dependencias escondidas")

- Tu proyecto usa la librería **A**.
- Pero **A** por dentro necesita **B**.
- Y **B** necesita **C**.




## ¿Por qué importa?

# 3.1 Ecosistemas (comparación rápida, sin memorizar)

| Lenguaje | Gestor de dependencias | Archivo donde se declara |
|---|---|---|
| Java | Maven / Gradle | `pom.xml` / `build.gradle` |
| JavaScript (Node) | npm / yarn | `package.json` |
| Python | pip / Poetry | `requirements.txt` / `pyproject.toml` |
| .NET | NuGet | `.csproj` |

*No necesitas dominar todos. En esta unidad trabajaremos fuerte con Maven (Java).*

# 3.1 Buenas prácticas (nivel principiante)

- No copies librerías a mano en carpetas raras.
- Declara dependencias en el archivo del proyecto.
- Usa versiones claras (no "cualquier versión").
- Si falla el build, comparte el mensaje de error **y** el archivo de dependencias.

**Nota**

"Mis dependencias no viven en mi PC, viven declaradas en el proyecto".

# Tarea 3.1 (sencilla, individual)

## Objetivo

Identificar dependencias en un proyecto y explicarlas en lenguaje simple.

1. Elige **una aplicación conocida** (WhatsApp, Instagram, Moodle, Spotify, etc.).
2. Escribe una lista de **5 "dependencias" imaginarias** que esa app necesitaría (ej. "librería para internet", "librería para notificaciones").
3. Para cada una, explica en 1 línea: **para qué sirve**.

*No es investigación técnica profunda: es para entrenar el concepto.*

# Foro 3.1 (discusión guiada)

## Pregunta

¿Por qué crees que "copiar librerías a mano" (por ejemplo, meter archivos en una carpeta `lib/`) puede causar problemas cuando trabajas en equipo?

- Responde con un ejemplo de la vida real (no técnico) y luego con un ejemplo de software.
- Comenta a 1 compañero: agrega una idea nueva, no repitas lo mismo.

# 3.2 ¿Por qué es importante? (4 razones claras)

- **Tiempo**: instalas todo con 1 comando, no a mano.
- **Calidad**: el proyecto construye igual en PC, laboratorio y servidor.
- **Equipo**: todos trabajan con las mismas versiones.
- **Seguridad**: si una librería tiene falla, sabes cuál versión usas.

**Ejemplo corto**

Si en una librería se descubre un problema, puedes actualizar solo esa versión y volver a construir.

# 3.2 ¿Qué es CI/CD? (sin miedo al acrónimo)

- **CI (Integración Continua)**: cada vez que subes cambios, se ejecuta automáticamente:
  - compilar,
  - correr pruebas,
  - avisar si algo se dañó.
- **CD (Entrega/Despliegue Continuo)**: además, puede:
  - generar el artefacto final,
  - y a veces desplegarlo en un servidor.

## Conexión con dependencias

Si no gestionas dependencias bien, CI/CD fallará aunque "en tu PC funcione".

# 3.2 Versiones: SemVer en ejemplo humano

- SemVer: `MAJOR.MINOR.PATCH` (ej. `2.4.1`)
- Piensa así:
  - **MAJOR** ($2 \rightarrow 3$): cambia reglas y puede romper tu código.
  - **MINOR** ($2.4 \rightarrow 2.5$): agrega cosas nuevas sin romper (ideal).
  - **PATCH** ($2.4.1 \rightarrow 2.4.2$): arregla errores.

**Regla práctica**

En proyectos reales, actualizar MAJOR requiere prueba extra y planificación.

# 3.2 Riesgos comunes por mala gestión de dependencias

- Conflictos de versiones (dos librerías piden versiones distintas).
- Dependencias transitivas con vulnerabilidades.
- Proyectos que no se pueden reconstruir meses después.
- "Bloqueo" del equipo porque nadie sabe qué versión usar.

**Señal roja**

Si alguien dice: "no sé por qué compila en mi PC", probablemente faltan dependencias declaradas.

# Tarea 3.2 (sencilla, individual)

## Objetivo

Entender por qué controlar versiones evita problemas.

1. Inventa un ejemplo simple: "Proyecto X usa una librería Y".
2. Escribe 3 versiones posibles: `1.0.0`, `1.1.0`, `2.0.0`.
3. Explica en 3 líneas:
   - ¿cuál versión actualizarías primero y por qué?
   - ¿cuál te daría más miedo y por qué?

# Foro 3.2 (discusión guiada)

## Pregunta

¿En qué situaciones reales vale la pena "congelar" versiones (no actualizar por un tiempo), y en qué situaciones vale la pena actualizar rápido?

- Da un ejemplo para un proyecto universitario y uno para una empresa.
- Responde a un compañero con una recomendación concreta.

# 3.3 ¿Qué es gestión integrada de proyectos? (en simple)

- Es usar herramientas para que el proyecto sea **visible** y **ordenado**.
- Te ayuda a responder preguntas típicas:
  - ¿Qué se está haciendo hoy?
  - ¿Qué falta?
  - ¿Quién es responsable?
  - ¿Qué evidencia existe del avance?

**Ejemplo**

Un tablero tipo "Pendiente → En progreso → En revisión → Hecho".

# 3.3 ¿Cómo ayuda con el cliente? (lo que el cliente quiere ver)

- **Claridad**: tareas claras, sin ambigüedad.
- **Evidencia**: enlaces, capturas, entregables.
- **Prioridad**: qué se hará primero.
- **Aceptación**: cuándo algo está "listo" (criterios de aceptación).

**Criterios de aceptación (en una línea)**

Reglas simples para decir: "sí, esto está terminado y cumple".

# 3.3 RBAC (roles y permisos) explicado fácil

- RBAC = **Role-Based Access Control** (control por roles).
- En español: "no todos pueden hacer todo".

## Ejemplo

- **Docente/Administrador**: crea y edita tareas.
- **Estudiantes**: actualizan tareas asignadas.
- **Cliente (simulado)**: solo ve y comenta (no borra nada).

*Esto evita accidentes y mantiene orden.*

# 3.3 Herramientas típicas (no hay que casarse con una)

- GitHub Projects
- Trello (muy visual)
- Jira / Azure DevOps (más completo, más "empresa")

**En esta unidad**

Lo importante no es la marca, sino el **proceso**: tareas claras + evidencia + seguimiento + aceptación.

# 3.3 Checklist para evaluar una herramienta (modo estudiante)

| **Criterio** | **¿Qué debo mirar?** |
|---|---|
| Facilidad | ¿Puedo usarla sin tutorial de 2 horas? |
| Roles | ¿Puedo dar acceso "solo comentar" a un cliente? |
| Evidencia | ¿Puedo adjuntar enlaces, imágenes, archivos? |
| Seguimiento | ¿Veo qué está pendiente y qué está listo? |
| Integración | ¿Se conecta con el repo (issues/commits)? |

# 3.3 Práctica guiada (mini-flujo con cliente simulado)

1. Crear un repositorio en GitHub (o usar uno existente).
2. Activar un tablero (GitHub Projects) con 4 columnas:
   - Pendiente, En progreso, En revisión, Hecho
3. Crear 5 tareas con:
   - descripción simple (qué y para qué),
   - criterios de aceptación (1 o 2 reglas).
4. Asignar un "cliente" (compañero) que solo comente.
5. Cada tarea terminada debe tener evidencia: enlace, captura o commit.

# Tarea 3.3 (sencilla, por parejas)

## Objetivo

Simular comunicación cliente–equipo con un tablero simple.

1. Creen un tablero con 4 columnas (Pendiente/Progreso/Revisión/Hecho).
2. Creen 5 tareas de un proyecto ficticio (ej. “Sistema de turnos”, “App de pedidos”).
3. Para cada tarea pongan **1 criterio de aceptación** (una frase).
4. Un estudiante hará de **cliente** y comentará 2 tareas (pregunta o sugerencia).

*No es programar: es organizar y comunicar.*

# Foro 3.3 (discusión guiada)

## Pregunta

¿Qué información mínima debe ver un cliente cada semana para confiar en el avance de un proyecto?

- Escribe una lista de 5 elementos (ej. tareas hechas, tareas en riesgo, fecha próxima entrega).
- Comenta a un compañero: ¿qué elemento le falta o le sobra y por qué?

# ¿Qué es Maven? (en palabras simples)

- Maven es una herramienta para proyectos Java que ayuda a:
  - **descargar dependencias** automáticamente,
  - **construir** el proyecto (compilar, probar, empaquetar),
  - generar un archivo final (artefacto), por ejemplo un `.jar`.

**¿Por qué aparece en Gestión de Configuración?**

Porque deja el proyecto **reproducible**: cualquiera puede construirlo con pasos estándar.

# POM: el archivo principal de Maven

- `pom.xml` es como el **"acta de nacimiento"** del proyecto.
- Ahí se define:
  - nombre del proyecto,
  - versión,
  - dependencias,
  - y reglas para construirlo.

**Traducción rápida**

POM = Project Object Model (modelo del proyecto). "el archivo donde se configura todo".

# GroupId y ArtifactId (con analogía)

- Maven identifica cosas con 3 datos:
  - **groupId** = "familia" o institución (ej. `ec.edu.ug`)
  - **artifactId** = "nombre" del proyecto (ej. `demo-maven`)
  - **version** = edición (ej. `1.0.0`)

**Ejemplo mental**

Como "Apellido + Nombre + Fecha": `ec.edu.ug:demo-maven:1.0.0`

# Ejemplo mínimo de pom.xml

```
<project>
  <modelVersion>4.0.0</modelVersion>

  <groupId>ec.edu.ug</groupId>
  <artifactId>demo-maven</artifactId>
  <version>1.0.0</version>

  <name>Demo Maven UG</name>
</project>
```

# Scope (alcance) de una dependencia: ¿cuándo se usa?

- El **scope** dice: "¿esta librería se usa para correr el programa o solo para pruebas?"

| Scope | Significa |
|---|---|
| `compile` | Se usa para compilar y para ejecutar (lo normal). |
| `test` | Solo se usa cuando corremos pruebas (no va a producción). |
| `provided` | El servidor/entorno ya lo trae (casos especiales). |

*Regla fácil: si es para pruebas, usa `test`.*

# Ejemplo: dependencia de pruebas (JUnit) con scope=test

```
<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

# Phase vs Goal (sin enredos)

- **Phase (fase)**: etapa del proceso (compile, test, package...).
- **Goal (meta)**: acción específica de un plugin.

## Ejemplos

- `mvn test` ejecuta la fase **test**.
- `mvn surefire:test` ejecuta un **goal** (del plugin Surefire).

*Como principiante, te basta con fases: `mvn test`, `mvn package`, etc.*

# Archetype: crear proyectos con plantilla

- Un **archetype** es una **plantilla** que genera un proyecto con estructura lista.
- Es como: "Nuevo proyecto → elegir plantilla → listo".

## Ventaja

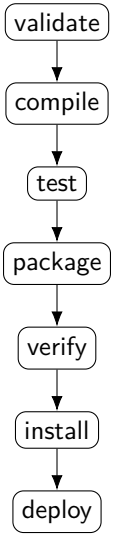
Evita que cada estudiante cree carpetas diferentes. Todos quedan con la misma estructura.

# Estructura típica Maven (¿dónde va cada cosa?)

| Ruta | Qué guardo ahí (en simple) |
|---|---|
| `pom.xml` | Reglas del proyecto (dependencias y construcción). |
| `src/main/java` | Código principal (lo que hace la app). |
| `src/test/java` | Pruebas unitarias (verificar funciones). |
| `target/` | Resultado del build (se crea solo). |

*No edites `target/`. Se borra y se vuelve a crear.*

# Ciclo de vida Maven (lo que pasa cuando construyes)



# ¿Qué hace cada fase? (traducción a español)

- `validate`: revisa que el proyecto está bien definido.
- `compile`: convierte el código Java en archivos `.class`.
- `test`: ejecuta pruebas (para detectar errores temprano).
- `package`: crea el archivo final (ej. `.jar`).
- `verify`: chequeos extra de calidad (si están configurados).
- `install`: guarda el artefacto en tu repositorio local.
- `deploy`: sube el artefacto a un repositorio remoto (empresa/universidad).

# Laboratorio 0: Lo que vamos a usar (sin complicarse)

- **VS Code** como editor.
- **JDK** (Java 17 o 21) para poder compilar.
- **Maven** para construir el proyecto.

**Nota**

Si hay problemas de instalación, lo más importante es que funcione: `java -version` y luego `mvn -version`.

# Laboratorio 0A (Windows): Comprobar Java y Maven

1. Abrir PowerShell.
2. Ejecutar:
   - `java -version`
   - `javac -version`
   - `mvn -version`
3. Si `java` no existe:
   - revisar variables: `JAVA_HOME` y `PATH`.

*Primero Java, después Maven.*

# Laboratorio 0B (Linux): Comprobar Java y Maven

1. En terminal:
   - `java -version`
   - `javac -version`
   - `mvn -version`
2. Si falta Maven, se instala con el gestor del sistema (según distro).

# Laboratorio 0C: VS Code para Java (pasos cortos)

1. Abrir VS Code.
2. Ir a **Extensions**.
3. Instalar: **Extension Pack for Java** (Microsoft).
4. Reiniciar VS Code.

## Señal de que todo está bien

VS Code detecta Java y ofrece opciones de ejecutar/pruebas en proyectos Java.

# Laboratorio 0D (Windows): PowerShell vs Git Bash

- **PowerShell**: viene con Windows (ok para comandos).
- **Git Bash**: se parece a Linux (útil para que comandos sean "iguales").

**Recomendación**

Si varios comandos del profesor son estilo Linux, usar Git Bash ayuda a que todos sigan igual.

# Laboratorio 1: Crear proyecto Maven (paso a paso)

1. Crear carpeta de trabajo:
   - Windows (PowerShell): `mkdir demo-maven-ug`
   - Linux/Git Bash: `mkdir demo-maven-ug`
2. Entrar a la carpeta:
   - Windows: `cd .\demo-maven-ug`
   - Linux/Git Bash: `cd demo-maven-ug`
3. Generar el proyecto con plantilla (archetype).

## Comando (igual para todos)

```
mvn archetype:generate -DgroupId=ec.edu.ug -DartifactId=demo-maven
-DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

# Laboratorio 1: Abrir en VS Code y probar

1. Entrar al proyecto: `cd demo-maven`
2. Abrir VS Code: `code .`
3. Ejecutar pruebas:
   - `mvn test`

**Esperado**

La primera vez tarda porque descarga dependencias. Luego aparece **BUILD SUCCESS**.

# Laboratorio 2: Agregar una dependencia (ejemplo simple)

1. Abrir `pom.xml`.
2. Buscar/crear `<dependencies>`.
3. Pegar una dependencia (ej. Gson).
4. Guardar.
5. Ejecutar: `mvn -q test`

**¿Qué debería pasar?**

Maven descarga la librería y el proyecto sigue funcionando.

# Laboratorio 2: Dependencia Gson (para leer JSON)

```
<dependencies>
  <dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.10.1</version>
  </dependency>
</dependencies>
```

# Laboratorio 3: Build completo (limpiar y empaquetar)

1. Ejecutar: `mvn clean package`
2. Revisar carpeta `target/`.
3. Buscar el `.jar` generado.

**¿Qué es `clean`?**

Borra `target/` para que el build sea "desde cero" y sea más confiable.

# Ejemplo mínimo de prueba JUnit (para entender `test`)

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class CalculadoraTest {
  @Test
  void suma_debe_funcionar() {
    int r = 2 + 3;
    assertEquals(5, r);
  }
}
```

# Maven Wrapper: ¿por qué ayuda?

- Todos usan **la misma versión de Maven** sin discutir instalaciones.
- Funciona en Windows y Linux.
- Reduce "a mí no me funciona" por diferencias de PC.

**Idea**

En vez de `mvn ...` usas:

- Linux: `./mvnw ...`
- Windows: `.\mvnw.cmd ...`

# Comandos con Wrapper (Windows vs Linux)

| **Windows (PowerShell)** | **Linux / Git Bash** |
|---|---|
| `.\mvnw.cmd -v` | `./mvnw -v` |
| `.\mvnw.cmd test` | `./mvnw test` |
| `.\mvnw.cmd clean package` | `./mvnw clean package` |

*Si vas a evaluar, pide Wrapper para que todos compilen igual.*

# Tarea Maven (sencilla, individual)

## Objetivo

Practicar el ciclo de vida sin complicaciones.

1. Crear el proyecto con el archetype (Laboratorio 1).
2. Ejecutar y capturar evidencia (pantallazo o copiar salida):
   - `mvn -version`
   - `mvn test`
   - `mvn clean package`
3. Escribir 5 líneas respondiendo:
   - ¿qué es `target/`?
   - ¿qué hace `test`?
   - ¿qué genera `package`?

# Foro Maven (discusión guiada)

## Pregunta

Si tu compañero dice: “mi proyecto no compila, pero en mi PC sí”, ¿qué 3 cosas revisarías primero?

- Responde con pasos concretos (no teoría).
- Comenta a un compañero con una mejora: “además revisaría...”.

# Buenas prácticas (lista corta y útil)

1. Declara dependencias (no copies librerías a mano).
2. Controla versiones (no "cualquier versión").
3. Construye seguido: `mvn test` y `mvn package`.
4. Si falla, comparte el error completo y el `pom.xml`.
5. Usa tablero para evidencias (tareas, comentarios, aceptación).

# Mini-resumen de la unidad

- **3.1** Dependencias = ingredientes del proyecto; se declaran y se descargan con una herramienta.
- **3.2** Importan para calidad, seguridad y trabajo en equipo (incluido CI/CD).
- **3.3** Gestión integrada = tablero + evidencias + roles para comunicarse con el cliente.
- **Maven** estandariza dependencias y construcción en Java.

# Fin

**¡Unidad 3!**